

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Программирование графических процессоров»

Классификация и кластеризация изображений на GPU.

Выполнил: *Д.Н. Путилин*

Группа: *8О-407Б*

Преподаватель: *А.Ю. Морозов*

Москва, 2024

Условие

1. Цель работы.

Научиться использовать GPU для классификации и кластеризации изображений.

Использование константной памяти и одномерной сетки потоков.

2. Вариант 2.

На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. На следующей строке, число nc - количество классов. Далее идут nc строчек описывающих каждый класс. В начале j -ой строки задается число nr_j -- количество пикселей в выборке, за ним следуют nr_j пар чисел -- координаты пикселей выборки. $nc \leq 32$, $nr_j \leq 2^{19}$, $w*h \leq 4*10^8$.

Для некоторого пикселя p , номер класса j_c определяется следующим образом:

$$j_c = \arg \max_j \left[- (p - avg_j)^T * cov_j^{-1} * (p - avg_j) \right]$$

Программное и аппаратное обеспечение

Compute capability : 7.5

Name : Tesla T4

Total Global Memory : 15835660288

Shared memory per block : 49152

Registers per block: 65536

Warp size : 32

Max threads per block : (1024, 1024, 64)

Max block : (2147483647, 65535, 65535)

Total constant memory : 65536

Multiprocessors count : 40

Метод решения

Для обработки множества пикселей входного изображения в каждом потоке будем использовать одномерную организацию потоков. Для этого преобразуем изображение в линейную форму. Каждый поток будет классифицировать пиксели, используя формулу $idx + offset$, где idx рассчитывается как $blockIdx.x * blockDim.x + threadIdx.x$ (базовый индекс текущего потока), а $offset$ равен $blockDim.x * gridDim.x$ (сдвиг для перехода к следующему набору пикселей).

Описание программы

Программа реализована в одном файле и включает две основные функции: `main` и `kernel`. Функция `main` отвечает за выполнение задач ввода, вывода и передачу данных в видеопамять. Функция `kernel` запускается на графическом процессоре и реализует вычисление классов для пикселей изображения согласно указанной формуле. Вектор средних значений и матрицы ковариации рассчитываются заранее и сохраняются в постоянной памяти для эффективного доступа. Для выполнения матричных операций

создадим отдельные структуры матриц размеров 1×3 , 3×1 , 3×3 и операции с ними, необходимые для выполнения алгоритма.

Результаты

Во всех тестах будем делить квадратное изображение на 30 классов (каждый класс определяется 30 пикселями), в таблице приведена сторона изображения. Время указано в миллисекундах. При замерах было исключено время на предподсчёт, выполняемый на процессоре.

	$3 \cdot 10^1$	10^2	$3 \cdot 10^2$	10^3	$3 \cdot 10^3$	10^4	$3 \cdot 10^4$
<512, 8>	0.31	1.83	17.82	161.11	661.81	6817.19	61704.1
<2048, 16>	0.17	0.98	8.63	91.39	443.00	3358.78	32153.2
<8192, 64>	0.28	0.46	4.19	45.32	283.38	1648.01	14296.5
<32768, 256>	0.93	0.95	4.75	45.01	257.83	1601.28	14108.1
CPU	13.86	151.73	1329.77	17214.3	148791	-	-

Рассмотрим работу программы на конкретном примере. Проведём разбиение на 4 класса.

Исходное и итоговое изображения следующие (для наглядности 1й класс окрашен в красный, 2й в зелёный, 3й в синий, 4й в жёлтый цвет).



Выводы

Во время выполнения лабораторной работы для задачи классификации и кластеризации изображения был применён графический процессор (GPU). Для повышения производительности использовалась константная память, которая хранила заранее вычисленные значения средних и ковариаций классов. Это позволило сократить затраты на вычисления во время выполнения. Для обработки изображения была организована одномерная сетка потоков, что упростило доступ к данным благодаря их линеаризации. Проведённые замеры времени выполнения продемонстрировали значительное превосходство GPU над CPU в скорости обработки.